initialize the activity and set the view layout—just what we need to render content from Hotwire Native.

As before, let Android Studio autocomplete the function call to automatically import `android.os.Bundle`.

ch01_16/android/app/src/main/java/com/masilotti/hikingjournal/MainActivity.kt

```
» import android.os.Bundle
  import dev.hotwire.navigation.activities.HotwireActivity

  class MainActivity : HotwireActivity() {
»     override fun onCreate(savedInstanceState: Bundle?) {
»         super.onCreate(savedInstanceState)
»     }
  }
```

Now that you have a bit of Swift experience under your belt, the `onCreate()` function shouldn't look too intimidating. Yes, Kotlin uses `fun` instead of `func` to declare the function. But I bet you can guess what the question mark after `Bundle` means! That's right, it's Kotlin's version of an `optional` in Swift, a *nullable type*.

Before setting the view, first declare the package at the top of the file. This tells Android Studio where to find non-code resources, like the XML layout file `activity_main.xml` you updated in the previous section.

ch01_17/android/app/src/main/java/com/masilotti/hikingjournal/MainActivity.kt

```
» package com.masilotti.hikingjournal

  import android.os.Bundle
  import dev.hotwire.navigation.activities.HotwireActivity

  class MainActivity : HotwireActivity() {
      override fun onCreate(savedInstanceState: Bundle?) {
          super.onCreate(savedInstanceState)
      }
  }
```